

Experiment 'p_13_3_recursive_reverse_string' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_13_3_recursive_reverse_string

Natural language statement: null

Method signature: p_13_3_recursive_reverse_string(text: string) returns (reversed: string)

Ensures

- `|reversed| == |text|`
- `forall i :: 0 <= i < |text| ==> reversed[i] == text[|text| - 1 - i]`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_13_3_recursive_reverse_string(text: string) returns (reversed: string)`

The method should respect the following contract:

`ensures |reversed| == |text|, ensures forall i :: 0 <= i < |text| ==> reversed[i] == text`
↪ `[|text| - 1 - i]`

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnOpengDIDFEQzELPZF1xrNjgHHVn

Model: gpt-5.5-2026-04-23

Created at: 1780665898

Finish reason: stop

Usage: 310 tokens in, and 5206 tokens out

```
{
  reversed := "";
  var i := 0;
  while i < |text|
    invariant 0 <= i <= |text|
    invariant |reversed| == i
    invariant forall j :: 0 <= j < i ==> reversed[j] == text[i - 1 - j]
  {
    assert 0 <= i < |text|;
    var prev := reversed;
    reversed := text[i..i+1] + prev;
    assert |text[i..i+1]| == 1;
    assert (text[i..i+1])[0] == text[i];
    assert |reversed| == i + 1;

    forall j | 0 <= j < i + 1
      ensures reversed[j] == text[i - j]
    {
      if j == 0 {
        assert reversed[j] == (text[i..i+1])[0];
      }
    }
  }
}
```

```

    assert (text[i..i+1])[0] == text[i];
    assert i - j == i;
    assert reversed[j] == text[i - j];
  } else {
    assert 0 <= j - 1 < i;
    assert reversed[j] == prev[j - 1];
    assert prev[j - 1] == text[i - 1 - (j - 1)];
    assert i - 1 - (j - 1) == i - j;
    assert reversed[j] == text[i - j];
  }
}

i := i + 1;
}
}

```

Final Program

```

method p_13_3_recursive_reverse_string(text: string) returns (reversed: string)
  ensures |reversed| == |text|
  ensures forall i :: 0 <= i < |text| ==> reversed[i] == text[|text| - 1 - i]
{
  reversed := "";
  var i := 0;
  while i < |text|
    invariant 0 <= i <= |text|
    invariant |reversed| == i
    invariant forall j :: 0 <= j < i ==> reversed[j] == text[i - 1 - j]
  {
    assert 0 <= i < |text|;
    var prev := reversed;
    reversed := text[i..i+1] + prev;
    assert |text[i..i+1]| == 1;
    assert (text[i..i+1])[0] == text[i];
    assert |reversed| == i + 1;

    forall j | 0 <= j < i + 1
      ensures reversed[j] == text[i - j]
    {
      if j == 0 {
        assert reversed[j] == (text[i..i+1])[0];
        assert (text[i..i+1])[0] == text[i];
        assert i - j == i;
        assert reversed[j] == text[i - j];
      } else {
        assert 0 <= j - 1 < i;
        assert reversed[j] == prev[j - 1];
        assert prev[j - 1] == text[i - 1 - (j - 1)];
        assert i - 1 - (j - 1) == i - j;
        assert reversed[j] == text[i - j];
      }
    }
  }
}

```

```
    }  
  }  
  
  i := i + 1;  
}  
}
```

Total Token Usage

Input tokens: 310

Output tokens: 5206

Reasoning tokens: 4778

Sum of ‘total tokens’: 5516

Experiment Timings

Overall Experiment started at 1780665895067, ended at 1780666045546, lasting 150479ms (150.48 seconds)

Iteration #1 started at 1780665895068, ended at 1780666045546, lasting 150478ms (150.48 seconds)